

145. La même fonction, trois fois

Une fonction s'écrit une fois et s'appelle autant qu'on veut : trois bandes, trois longueurs, trois lettres.

1. Ce qu'il faut comprendre

C'est le programme d'avant, avec deux appels de plus dans `main()`. La fonction, elle, ne change pas d'une lettre.

Chaque appel remplit les paramètres autrement : `bande(2, 7, ALPHABET[1], 6)` met 2 dans `colonne`, 7 dans `ligne`, le B dans `tuile`, 6 dans `longueur`. Six B, des colonnes 2 à 7.

C'est tout l'intérêt d'une fonction : le geste (« poser une rangée ») est écrit **une seule fois**. C'est aussi ce qui va nous donner envie de la **ranger à part** : une fonction aussi utile, on voudrait la réemployer dans d'autres programmes, sans la recopier.

Essaie : ajoute `bande(0, 11, ALPHABET[3], 20);` — une ligne entière de D, de la colonne 0 à la 19.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **949 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
démarrage      49 o ← la console, à l'allumage - la mise en route de la console
bande()         38 o ← main() - une fonction fournie par le langage
main()         64 o ← démarrage - ta fonction
VBlank          8 o ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage   28 o ← image() - attend l'image suivante
AttendreVBlank  16 o ← démarrage - attend que l'écran soit libre
AdresseCase     10 o ← poser() - trouve l'adresse d'une case de l'écran
CopierTuiles    18 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte    14 o ← démarrage - vide les deux cartes de l'écran
Tuiles         704 o ← CopierTuiles - la police entière
Lettres gravées : toutes (poser() reçoit un numéro calculé)
Pas gravé (jamais appelé) : AvancerAirs, AttendreAcces, AttendreFinVBlank, EffacerCases, EcrireTexte, EcrireNombre,
Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond,
EffacerPanneau, LireManette, DonneesTransfert, Notes
Fonctions du langage appelées :
poser() × 1      17 o sur place, dans bande() + AdresseCase (10 o)
image() × 1       3 o sur place, dans main() + AttendreImage (28 o)
Tes fonctions appelées :
bande() × 3      57 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// La même fonction, trois fois : seuls les appels de main() changent.

#include <poser>      // pose une tuile sur une case du fond
#include <ALPHABET>   // ALPHABET[0] = A, [1] = B, [2] = C...

// bande(colonne, ligne, tuile, longueur) : comme à l'étape d'avant.
```

2. UNE FONCTION — BANDE()

```
void bande(uint8_t colonne, uint8_t ligne, uint8_t tuile, uint8_t longueur) {
    for (uint8_t i = 0; i < longueur; i++) {
        poser(colonne + i, ligne, tuile);
    }
}
```

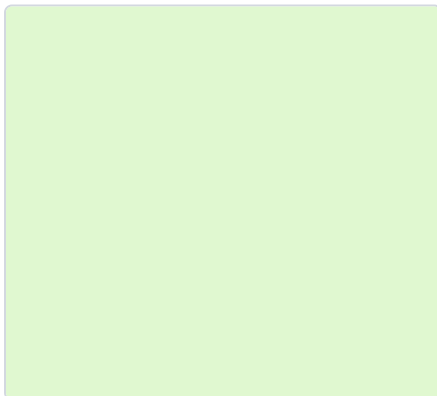
3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    bande(2, 5, ALPHABET[0], 10); // dix A : colonnes 2 à 11, ligne 5
    bande(2, 7, ALPHABET[1], 6);  // NOUVEAU : six B : colonnes 2 à 7, ligne 7
    bande(2, 9, ALPHABET[2], 3);  // NOUVEAU : trois C : colonnes 2 à 4, ligne 9

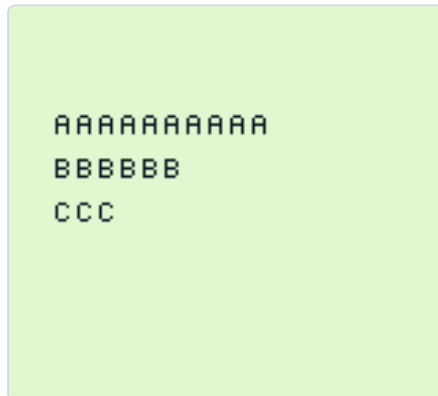
    while (true) {
        image();
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Trois rangées : dix A, six B, trois C, toutes calées à gauche sur la colonne 2.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ dix A sur la ligne 5

✓ six B sur la ligne 7

✓ trois C sur la ligne 9

Ensuite : **146. Ranger sa fonction dans un fichier voisin : `#include "outils.cpp"`** — chapitre 25 : Tes propres `#include` : ajouter une fonction. · gameboy3 — écrire une cartouche Game Boy en C++